

UNIT-VI DIGITAL ELECTRONICS**INTRODUCTION:**

Electronic circuits and systems are of two kinds. They are **analog and digital**. The distinction between them is not so much in the types of semiconductor devices used in these circuits as it is in voltage and current variations that occur when each type of circuit performs the function for which it is designed.

Analog circuits are those in which voltages and currents vary continuously through the given range. They can take infinite values within the specified range.

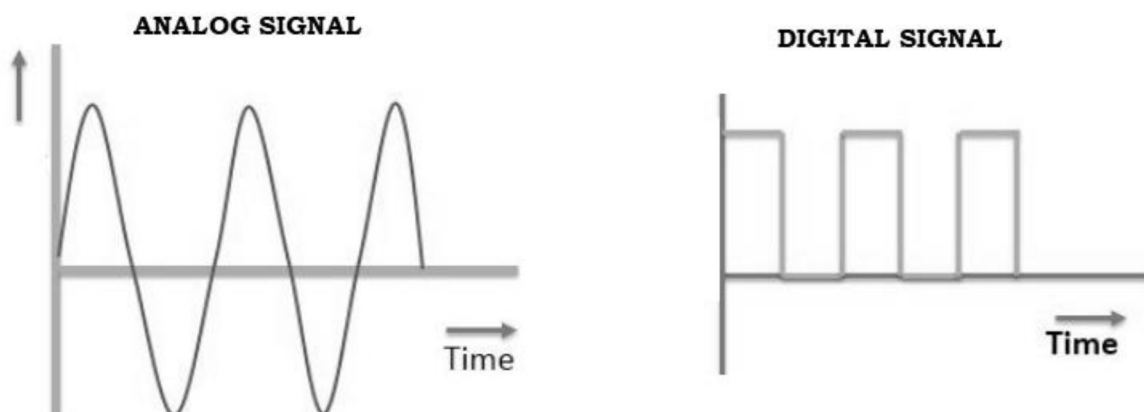
Digital circuit is one in which the voltage levels assume a finite number of distinct values. In virtually all modern digital circuits, there are just two discrete voltage levels.

Digital circuits are also called logic circuits, because each type of digital circuit obeys a certain set of logic rules. The manner in which a logic circuit responds to an input is referred to as the circuit's logic.

Digital systems are used extensively in computation and data processing, control systems, communications and measurement. Digital systems have a number of advantages over analog systems.

ADVANTAGES OF DIGITAL SYSTEMS OVER ANALOG SYSTEMS

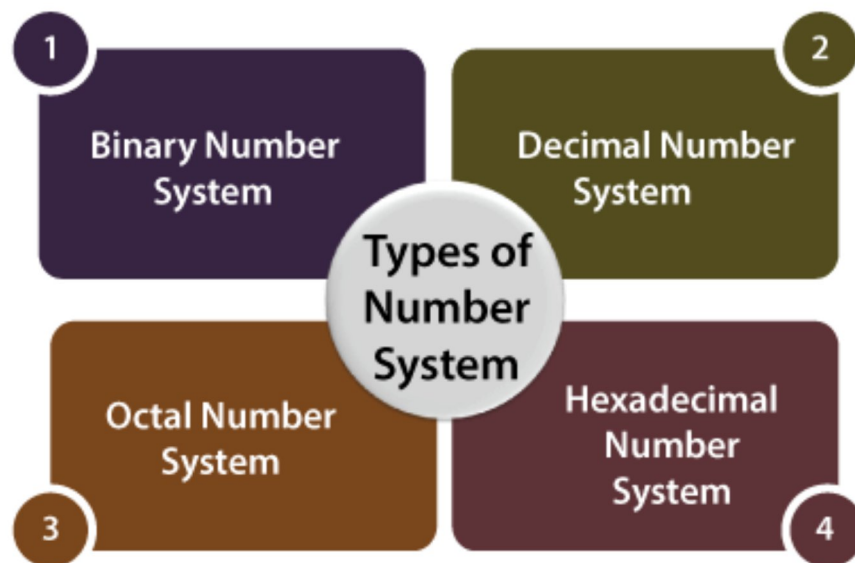
- ✓ Digital systems are easier to design
- ✓ These are small in size and simple operation (LOW or HIGH)
- ✓ Information storage is easy
- ✓ Accuracy and precision are greater
- ✓ Digital systems are more versatile
- ✓ Digital circuits are less affected by noise, temperature etc.,
- ✓ Reliability is more
- ✓ Requires less space & cost is low (Chips).
- ✓ Speed is high.

Example:

Applications of Digital systems:

- ✓ Cameras
- ✓ Recorders(Audio & Video)
- ✓ Telephone systems
- ✓ Traffic Lights
- ✓ Computers, TVs, DVDs etc.,
- ✓ Elevator Display
- ✓ Digital thermo meter
- ✓ Digital watch

TYPES OF NUMBER SYSTEMS



- i) Decimal Number System
- ii) Binary Number System
- iii) Octal Number System
- iv) Hexadecimal Number System

Table: Numbersystemandtheir Base value

Number Systems		
System	Base	Digits
Binary	2	0,1
Octal	8	0 1 2 3 4 5 6 7
Decimal	10	0 1 2 3 4 5 6 7 8 9
Hexadecimal	16	0 1 2 3 4 5 6 7 8 9 A B C D E F

Decimal Number System

The decimal number system contains ten unique symbols 0,1,2,3,4,5,6,7,8 and 9.

In decimal system 10 symbols are involved, so the **base or radix is 10**.

The decimal number system contains ten digits from 0 to 9 (base 10).

The position in the decimal number system specifies the power of the base (10). The 0 is the minimum value of the digit, and 9 is the maximum value of the digit.

Example:

When we write a decimal number say **5678.9**, we know it can be represented as

$$5000+600+70+8+0.9=5678.9$$

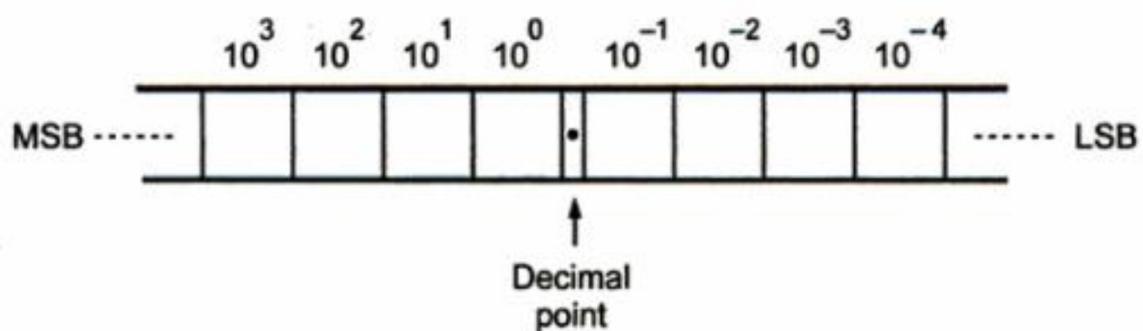
The decimal number 5678.9 can also be written as $(5678.9)_{10}$, where the 10 subscript indicates the radix or base.

In power of 10, we can write as

$$\boxed{5 \times 10^3} + \boxed{6 \times 10^2} + \boxed{7 \times 10^1} + \boxed{8 \times 10^0} + \boxed{9 \times 10^{-1}}$$

5 6 7 8 . 9

This says that, the position of a digit with reference to the decimal point determines its value/weight. The sum of all the digits multiplied by their weights gives the total number being represented. The leftmost digit, which has the greatest weight is called the most significant digit/bit (MSD or MSB) and the rightmost digit, which has the least weight, is called the least significant digit/bit (LSD or LSB).



EX: 9256.26

$$\begin{aligned} 9256.26 &= 9 \times 1000 + 2 \times 100 + 5 \times 10 + 6 \times 1 + 2 \times (1/10) + 6 \times (1/100) \\ &= 9 \times 10^3 + 2 \times 10^2 + 5 \times 10^1 + 6 \times 10^0 + 2 \times 10^{-1} + 6 \times 10^{-2} \end{aligned}$$

Binary Number system

A binary number system is used in the digital computers. In this number system, it carries only two digits, either 0 or 1.

Octal Number System

The octal number system has base 8 (means it has only eight digits from 0 to 7). An octal number system carries eight digits starting from 0, 1, 2, 3, 4, 5, 6, and 7. It is also known as the base 8 number system.

The position of a digit represents the 0 power of the base(8).

Example: 80

The position of the last digit represents the x power of the base(8). Example: 8^x , where x represents the last position, i.e., 1

Number	Octal Number (421)
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Hexadecimal Number System

The number system has a base of 16 means there are total 16 symbols(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F) used for representing a number.

The single-bit representation of decimal values 10, 11, 12, 13, 14, and 15 are represented by A, B, C, D, E, and F.

Only 4 bits are required for representing a number in a hexadecimal number.

Each set of bits has a distinct value between 0 and 15.

It has ten digits from 0 to 9 and 6 letters from A to F.

The letters from A to F defines numbers from 10 to 15.

It is also known as the base 16 number system.

Number	Hexa Decimal Number (8421)
--------	-------------------------------

Examples:

$(FAC2)_{16}$, $(564)_{16}$,
 $(11F3)_{16}$.

$(0ABD5)_{16}$, $(1123)_{16}$,

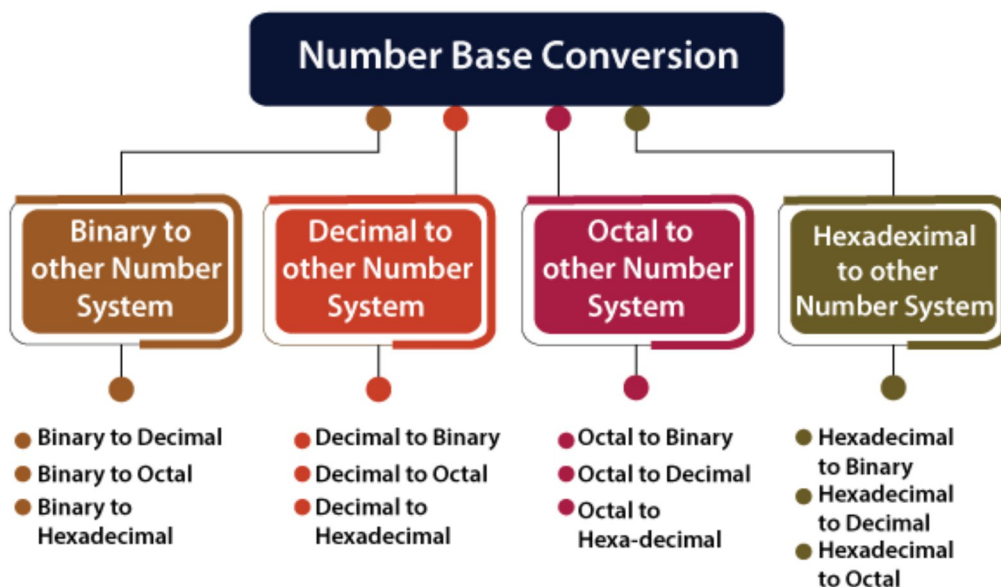
CONVERSION FROM SYSTEM TO

1. Binary to other Number
2. Decimal to other
3. Octal to other Number
4. Hexadecimal to other

ONE NUMBER ANOTHER

Systems.
 Number Systems.
 Systems.
 Number Systems.

0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10(A)	1010
11(B)	1011
12(C)	1100
13(D)	1101
14(E)	1110
15(F)	1111



Binary-To-Decimal Conversion:

1) Convert $(10101)_2$ to decimal

$$\begin{aligned}
 &\text{Positional weights } 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0 \\
 &\text{Binary no. } 10101_2 = (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\
 &\quad = 16 + 0 + 4 + 0 + 1 \\
 &\quad = 21_{10}
 \end{aligned}$$

2) Convert $(11011.101)_2$ to decimal

$$\begin{array}{cccccccc}
2^4 & 2^3 & 2^2 & 2^1 & 2^0 & 2^{-1} & 2^{-2} & 2^{-3} \\
1 & 1 & 0 & 1 & 1 & 1 & 0 & 1
\end{array}
= (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\
+ (1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3}) \\
= 16 + 8 + 0 + 2 + 1 + 0.5 + 0 + 0.125 \\
= 27.625_{10}$$

(ii) Convert $(111.101)_2$ to decimal.

Solution:

$$\begin{aligned}
(111.101)_2 &= (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3}) \\
&= 4 + 2 + 1 + 0.5 + 0 + 0.125 \\
&= (7.625)_{10}
\end{aligned}$$

Example 1: $(10110.001)_2$

We multiplied each bit of $(10110.001)_2$ with its respective positional weight, and last we add the products of all the bits with its weight.

$$(10110.001)_2 = (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) + (0 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3})$$

$$(10110.001)_2 = (1 \times 16) + (0 \times 8) + (1 \times 4) + (1 \times 2) + (0 \times 1) + (0 \times 1/2) + (0 \times 1/4) + (1 \times 1/8)$$

$$(10110.001)_2 = 16 + 0 + 4 + 2 + 0 + 0 + 0 + 0.125$$

$$(10110.001)_2 = (22.125)_{10}$$

Decimal to binary conversion

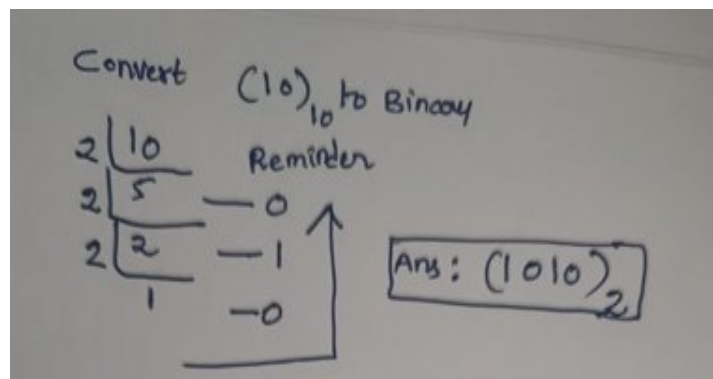
Steps

1. Divide the decimal number by 2
2. Write down the remainder (0 or 1)
3. Divide the quotient again by 2
4. Repeat until quotient becomes 0
5. Write the remainders from bottom to top → Binary number

Example 1: Convert $(10)_{10}$ to Binary

Division	Quotient	Remainder
$10 \div 2$	5	0
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1

Binary = $(1010)_2$



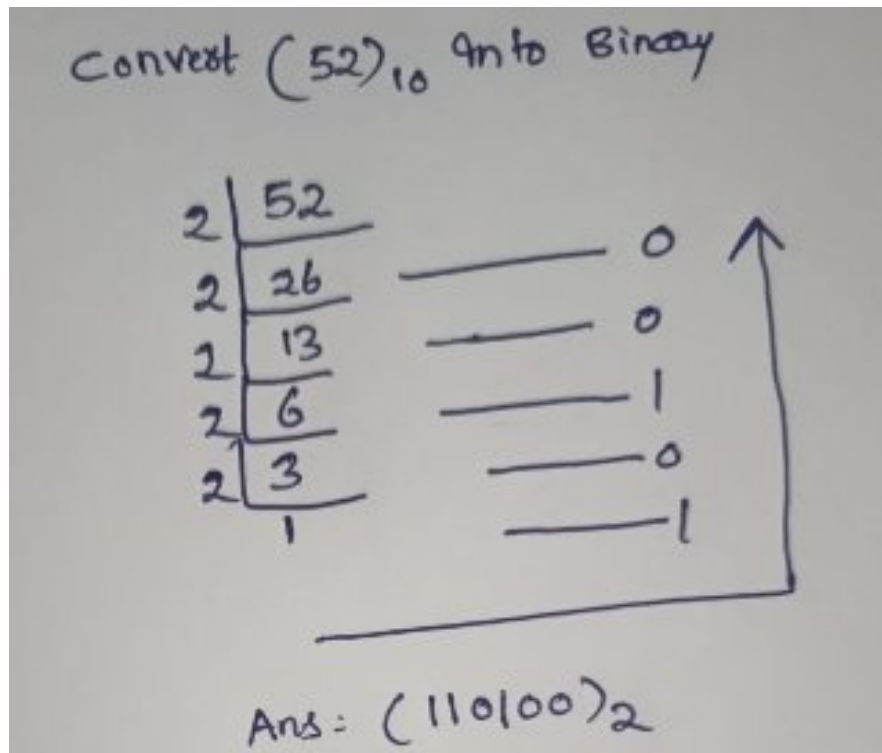
Convert 52_{10} to binary

Successive division

Remainder

2	52	
2	26	0
2	13	0
2	6	1
2	3	↑ 0
2	1	1
	0	1

Reading the remainders from bottom to top, the result is $52_{10} = 110100_2$.



Convert $(52.15)_{10}$ into Binary

Step 1: Convert Integer Part $(52)_{10}$ to Binary

Use repeated division by 2.

Division	Quotient	Remainder
$52 \div 2$	26	0
$26 \div 2$	13	0
$13 \div 2$	6	1
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1

Read remainders bottom to top

$(52)_{10} = (110100)_2$

Step 2: Convert Fractional Part $(0.15)_{10}$ to Binary

Use repeated multiplication by 2.

Multiplication	Result	Integer Part
0.15×2	0.30	0

Multiplication	Result	Integer Part
0.30×2	0.60	0
0.60×2	1.20	1
0.20×2	0.40	0
0.40×2	0.80	0
0.80×2	1.60	1
0.60×2	1.20	1
0.20×2	0.40	0

Take integer parts in order

$$(0.15)_{10} \approx (0.00100110)_2$$

(fractional part is non-terminating, so we stop after 8 bits)

Step 3: Combine Integer and Fractional Parts

$$(52.15)_{10} = (110100.00100110)_2$$

Final Answer

$$(52.15)_{10} \approx (110100.00100110)_2$$

Convert $(105)_{10}$ to Binary

2	105	
2	52	— 1
2	26	— 0
2	13	— 0
2	6	— 1
2	3	— 0
	1	— 1

↑

Ans: $(1101001)_2$

Convert 105.15_{10} to binary

Conversion of integer 105

Successive division

Remainder

2	105	
2	52	1
2	26	0
2	13	0
2	6	1
2	3	↑ 0
2	1	1
	0	1

Reading the remainders from bottom to top, $105_{10} = 1101001_2$.

Conversion of fraction 0.15_{10}

<i>Given fraction</i>	0.15
Multiply 0.15 by 2	0.30
Multiply 0.30 by 2	0.60
Multiply 0.60 by 2	1.20
Multiply 0.20 by 2	↓ 0.40
Multiply 0.40 by 2	0.80
Multiply 0.80 by 2	1.60

This particular fraction can never be expressed exactly in binary. This process may be terminated after a few steps.

Reading the integers from top to bottom, $0.15_{10} = 0.001001_2$.

Therefore, the final result is, $105.15_{10} = 1101001.001001_2$.

Convert $(7)_{10}$ to Binary

Division	Quotient	Remainder
$7 \div 2$	3	1
$3 \div 2$	1	1
$1 \div 2$	0	1

$(7)_{10} = (111)_2$

Convert $(18)_{10}$ to Binary

Division	Quotient	Remainder
$18 \div 2$	9	0
$9 \div 2$	4	1
$4 \div 2$	2	0
$2 \div 2$	1	0
$1 \div 2$	0	1

$$(18)_{10} = (10010)_2$$

Convert decimal number $(152.25)_{10}$ into binary

Divide the number 152 and its successive quotients with base 2.

Operation	Quotient	Remainder
$152/2$	76	0 (LSB)
$76/2$	38	0
$38/2$	19	0
$19/2$	9	1
$9/2$	4	1
$4/2$	2	0
$2/2$	1	0
$1/2$	0	1(MSB)

$$(152)_{10} = (10011000)_2$$

BINARY TO OCTAL CONVERAION

Steps

1. Start grouping binary digits **from right to left**
2. Make groups of **3 bits**
3. If needed, add **leading zeros**
4. Convert each 3-bit group into its octal value
5. Write the octal digits together

Binary → Octal Table (Must Remember)

Binary	Octal
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Example 1

Convert $(110100)_2$ to Octal

Step 1: Group into 3 bits

110 100

Step 2: Convert each group

- $110 \rightarrow 6$
- $100 \rightarrow 4$

Answer:

$$(110100)_2 = (64)_8$$

Example 2

Convert $(1010111)_2$ to Octal

Step 1: Group from right

001 010 111

Step 2: Convert

- $001 \rightarrow 1$

- $010 \rightarrow 2$
- $111 \rightarrow 7$

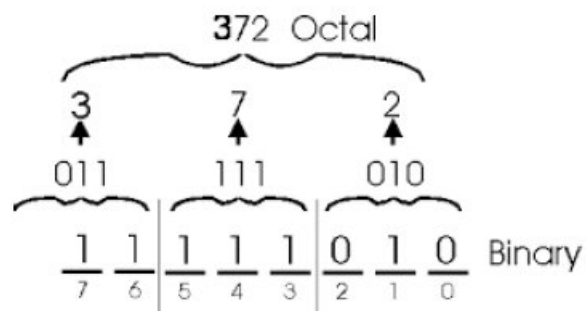
Answer:

$$(1010111)_2 = (127)_8$$

Binary to octal:

Example: $100\ 111010_2 = (100)(111)(010)_2 = 4\ 7\ 2_8$

Octal to Binary:



Binary to Octal with Fraction

Rule

- Integer part $\rightarrow$ group from **right to left**
- Fraction part $\rightarrow$ group from **left to right**

Example : Convert $(110101.1011)_2$ to Octal

Step 1: Group integer part

110 101

Step 2: Group fractional part

101 100

(add one 0 to complete 3 bits)

Step 3: Convert

- $110 \rightarrow 6$
- $101 \rightarrow 5$
- $.101 \rightarrow 5$
- $.100 \rightarrow 4$

Answer:

$$(110101.1011)_2 = (65.54)_8$$

OCTAL TO BINARY CONVERSION

Steps

1. Write each octal digit separately
2. Replace each octal digit with its 3-bit binary equivalent
3. Combine all the binary groups

Octal → Binary Table (Must Remember)

Octal	Binary(421)
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Example 1

Convert $(57)_8$ to Binary

Step 1: Write binary of each digit

$5 \rightarrow 101$

$7 \rightarrow 111$

Step 2: Combine

Answer:

$$(57)_8 = (101111)_2$$

Example 2

Convert $(124)_8$ to Binary

1 → 001

2 → 010

4 → 100

Answer:

$$(124)_8 = (001010100)_2$$

(Leading zeros can be removed → 1010100_2)

Octal to Binary with Fraction

Rule

- Integer part → convert normally
- Fraction part → convert digit by digit after decimal point

Example 3

Convert $(36.5)_8$ to Binary

3 → 011

6 → 110

.5 → .101

Answer:

$$(36.5)_8 = (011110.101)_2$$

(Final form → 11110.101_2)

Example 4

Convert $(7.24)_8$ to Binary

7 → 111

.2 → .010

.4 → .100

Answer:

$$(7.24)_8 = (111.010100)_2$$

Each octal number converts to 3 binary digits

Code
0 - 000
1 - 001
2 - 010
3 - 011
4 - 100
5 - 101
6 - 110
7 - 111

To convert 653_8 to binary, just substitute code:

6	5	3
↓	↓	↓
110	101	011

BINARY TO HEXA DECIMAL NUMBERS

Hexadecimal → Binary Table

Hex	Binary(8421)
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Simple Method

1. Take each hexadecimal digit **one by one**
2. Replace it with its **4-bit binary equivalent**
3. Combine all binary groups

Example 1

Convert $(2F)_{16}$ to Binary

$2 \rightarrow 0010$

$F \rightarrow 1111$

Answer:

$$(2F)_{16} = (00101111)_2$$

(Leading zeros can be removed $\rightarrow 101111_2$)

Example 2

Convert $(A3)_{16}$ to Binary

$A \rightarrow 1010$

$3 \rightarrow 0011$

Answer:

$$(A3)_{16} = (10100011)_2$$

Example 3

Convert $(7B9)_{16}$ to Binary

$7 \rightarrow 0111$

$B \rightarrow 1011$

$9 \rightarrow 1001$

Answer:

$$(7B9)_{16} = (011110111001)_2$$

Hexadecimal to Binary with Fraction

Example 4

Convert $(A.5)_{16}$ to Binary

$A \rightarrow 1010$

$.5 \rightarrow .0101$

Answer:

$$(A.5)_{16} = (1010.0101)_2$$

Example 5

Convert $(3C.2F)_{16}$ to Binary

$3 \rightarrow 0011$

$C \rightarrow 1100$

$.2 \rightarrow .0010$

$F \rightarrow 1111$

Answer:

$$(3C.2F)_{16} = (00111100.00101111)_2$$

HEXA DECIMAL NUMBERS TO BINARY

Simple Conversion Method

1. Take each hexadecimal digit one by one
2. Replace it with its 4-bit binary equivalent
3. Combine all binary groups
4. Remove unnecessary leading zeros (optional)

Example 1

Convert $(A)_{16}$ to Binary

$A \rightarrow 1010$

Answer:

$$(A)_{16} = (1010)_2$$

Example 2

Convert $(2F)_{16}$ to Binary

$2 \rightarrow 0010$

$F \rightarrow 1111$

Combine $\rightarrow 00101111$

Answer:

$$(2F)_{16} = (00101111)_2$$

Example 3**Convert $(9C)_{16}$ to Binary** $9 \rightarrow 1001$ $C \rightarrow 1100$ Combine $\rightarrow$ **10011100****Answer:**

$$(9C)_{16} = (10011100)_2$$

Example 4**Convert $(7B9)_{16}$ to Binary** $7 \rightarrow 0111$ $B \rightarrow 1011$ $9 \rightarrow 1001$ Combine $\rightarrow$ **011110111001****Answer:**

$$(7B9)_{16} = (011110111001)_2$$

Hexadecimal to Binary with Fraction**Example 5****Convert $(A.5)_{16}$ to Binary** $A \rightarrow 1010$ $.5 \rightarrow .0101$ **Answer:**

$$(A.5)_{16} = (1010.0101)_2$$

Example 6**Convert $(3C.2F)_{16}$ to Binary** $3 \rightarrow 0011$ $C \rightarrow 1100$ $.2 \rightarrow .0010$ $F \rightarrow .1111$ Combine $\rightarrow$ **00111100.00101111****Answer:**

$$(3C.2F)_{16} = (00111100.00101111)_2$$

Decimal to Octal Conversion

Decimal to octal conversion means changing a **base-10 number** into a **base-8 number**.

Octal digits: 0, 1, 2, 3, 4, 5, 6, 7

Method: Repeated Division by 8

Steps

1. Divide the decimal number by 8
2. Write the **remainder**
3. Divide the quotient again by 8
4. Continue until quotient becomes 0
5. Read remainders from bottom to top

Example 1

Convert $(125)_{10}$ to Octal

Division	Quotient	Remainder
$125 \div 8$	15	5
$15 \div 8$	1	7
$1 \div 8$	0	1

Bottom to top $\rightarrow$ **175**

Answer:

$$(125)_{10} = (175)_8$$

Example 2

Convert $(64)_{10}$ to Octal

Division	Quotient	Remainder
$64 \div 8$	8	0
$8 \div 8$	1	0
$1 \div 8$	0	1

Answer:

$$(64)_{10} = (100)_8$$

Decimal to Octal with Fraction

Method: Repeated Multiplication by 8

Example 3

Convert $(12.625)_{10}$ to Octal

Integer Part (12)

Division	Quotient	Remainder
$12 \div 8$	1	4
$1 \div 8$	0	1

Integer part = 14

Fractional Part (0.625)

Multiplication Result Integer Part

$$0.625 \times 8 = 5.000 \quad 5$$

Fractional part = .5

Answer:

$$(12.625)_{10} = (14.5)_8$$

To convert the given decimal integer number to octal, successively divide the given number by 8 till the quotient is 0. To convert the given decimal fractions to octal successively multiply the decimal fraction and the subsequent decimal fractions by 8 till the product is 0 or till the required accuracy is obtained.

For example:

(i) Convert $(378.93)_{10}$ into octal.

Solution:

$$\begin{array}{r} 8 \overline{) 378} \\ 8 \overline{) 47} \quad - 2 \\ 8 \overline{) 5} \quad - 7 \\ \hline 0 \quad - 5 \end{array}$$

$$0.93 \times 8 = 7.44$$

$$0.44 \times 8 = 3.52$$

$$0.52 \times 8 = 4.16$$

$$0.16 \times 8 = 1.28$$

Result of $(378.93)_{10}$ is $(572.7341)_8$

9) Convert 378.93_{10} to Octal

Conversion of 378_{10} to octal

Successive division

8		378
8		47
8		5
		0

Remainders

↑	2
	7
	5

Read the remainders from bottom to top. Therefore, $378_{10} = 572_8$.

Conversion of 0.93_{10} to octal

0.93×8		7.44
0.44×8		3.52
0.52×8	↓	4.16
0.16×8		1.28

Read the integers to the left of the octal point downwards.
Therefore, $0.93_{10} = 0.7341_8$. Hence $378.93_{10} = 572.7341_8$.

Octal to Decimal Conversion

What is Octal to Decimal Conversion?

Octal to decimal conversion means converting a base-8 number into a base-10 number.

Steps

1. Write positional values of **powers of 8**
2. Multiply each digit with its positional value
3. Add all results

Example

Convert $(245)_8$ to Decimal

$$\begin{aligned} &= 2 \times 8^2 + 4 \times 8^1 + 5 \times 8^0 \\ &= 2 \times 64 + 4 \times 8 + 5 \times 1 \\ &= 128 + 32 + 5 = 165 \end{aligned}$$

Answer:

$$(245)_8 = (165)_{10}$$

Example

Convert $(73)_8$ to Decimal

$$\begin{aligned} &= 7 \times 8^1 + 3 \times 8^0 \\ &= 56 + 3 = 59 \end{aligned}$$

Answer:

$$(73)_8 = (59)_{10}$$

Octal to Decimal with Fraction

Example

Convert $(12.4)_8$ to Decimal

$$\begin{aligned} &= 1 \times 8^1 + 2 \times 8^0 + 4 \times 8^{-1} \\ &= 8 + 2 + 0.5 \\ &= 10.5 \end{aligned}$$

Answer:

$$(12.4)_8 = (10.5)_{10}$$

For conversion octal to decimal number, multiply each digit in the octal number by the weight of its position and add all the product terms

For example: -

Convert $(4057.06)_8$ to decimal

Solution:

$$\begin{aligned} (4057.06)_8 &= 4 \times 8^3 + 0 \times 8^2 + 5 \times 8^1 + 7 \times 8^0 + 0 \times 8^{-1} + 6 \times 8^{-2} \\ &= 2048 + 0 + 40 + 7 + 0 + 0.0937 \\ &= (2095.0937)_{10} \end{aligned}$$

Result is $(2095.0937)_{10}$

3) Convert 4057.06_8 to decimal

$$\begin{aligned}4057.06_8 &= 4 \times 8^3 + 0 \times 8^2 + 5 \times 8^1 + 7 \times 8^0 + 0 \times 8^{-1} + 6 \times 8^{-2} \\&= 2048 + 0 + 40 + 7 + 0 + 0.0937 \\&= 2095.0937_{10}\end{aligned}$$

4) Convert 172_8 to decimal

$$\begin{array}{ccc}1 & 7 & 2 \\8^2 & 8^1 & 8^0 \\ \text{Weight} & = & 1 \times 8^2 + 7 \times 8^1 + 2 \times 8^0 \\& = & 1 \times 64 + 7 \times 8 + 2 \times 1 \\& = & 122_{10}\end{array}$$

Decimal to Hexadecimal Conversion

What is Decimal to Hexadecimal Conversion?

Decimal to hexadecimal conversion means changing a **base-10 number** into a **base-16 number**.

Hexadecimal digits:

0–9 and A=10, B=11, C=12, D=13, E=14, F=15

Method: Repeated Division by 16

Steps

1. Divide the decimal number by 16
2. Write the remainder
3. Convert remainder 10–15 into A–F
4. Divide the quotient again by 16
5. Continue until quotient becomes 0
6. Read remainders from bottom to top

Example 1

Convert $(45)_{10}$ to Hexadecimal

Division	Quotient	Remainder
$45 \div 16$	2	$13 \rightarrow D$
$2 \div 16$	0	2

Bottom to top $\rightarrow$ **2D**

Answer:

$$(45)_{10} = (2D)_{16}$$

Example 2

Convert $(254)_{10}$ to Hexadecimal

Division	Quotient	Remainder
$254 \div 16$	15	$14 \rightarrow E$
$15 \div 16$	0	$15 \rightarrow F$

Bottom to top $\rightarrow$ **FE**

Answer:

$$(254)_{10} = (FE)_{16}$$

Decimal to Hexadecimal with Fraction

Method: Repeated Multiplication by 16

Example 3

Convert $(26.75)_{10}$ to Hexadecimal

Integer part (26)

Division	Quotient	Remainder
$26 \div 16$	1	$10 \rightarrow A$
$1 \div 16$	0	1

Integer part = **1A**

Fractional part (0.75)

Multiplication	Result	Integer Part
0.75×16	12.00	C

Fractional part = **.C**

Final Answer:

$$(26.75)_{10} = (1A.C)_{16}$$

10) Convert 2598.675_{10} to Hexadecimal

Conversion of 2598_{10}

Successive division

16	2598
16	162
16	10
	0

<i>Remainder</i>	
<i>Decimal</i>	<i>Hex</i>
6	↑ 6
2	2
10	A

Reading the remainders upwards, $2598_{10} = A26_{16}$.

Conversion of 0.675_{10}

Given fraction is 0.675

0.675×16	10.8
0.800×16	12.8
0.800×16	12.8
0.800×16	↓ 12.8

Reading the integers to the left of hexadecimal point downwards, $0.675_{10} = 0.ACCC_{16}$.

Therefore, $2598.675_{10} = A26.ACCC_{16}$.

Hexadecimal to Decimal Conversion

What is Hexadecimal to Decimal Conversion?

Hexadecimal to decimal conversion means converting a base-16 number into a base-10 number.

Method: Positional Value Method

Steps

1. Write positional values of **powers of 16**
2. Replace A–F with **10–15**
3. Multiply and add all values

Example 4

Convert $(2D)_{16}$ to Decimal

$$\begin{aligned}
 &= 2 \times 16^1 + D \times 16^0 \\
 &= 2 \times 16 + 13 \times 1 \\
 &= 32 + 13 = 45
 \end{aligned}$$

Answer:

$$(2D)_{16} = (45)_{10}$$

Example 5

Convert $(A3)_{16}$ to Decimal

$$\begin{aligned}
 &= A \times 16^1 + 3 \times 16^0 \\
 &= 10 \times 16 + 3 \\
 &= 160 + 3 = 163
 \end{aligned}$$

Answer:

$$(A3)_{16} = (163)_{10}$$

Hexadecimal to Decimal with Fraction

Example 6

Convert $(1F.8)_{16}$ to Decimal

$$\begin{aligned}
 &= 1 \times 16^1 + F \times 16^0 + 8 \times 16^{-1} \\
 &= 16 + 15 + 0.5 \\
 &= 31.5
 \end{aligned}$$

Answer:

$$(1F.8)_{16} = (31.5)_{10}$$

Hexadecimal to Octal Conversion

Steps (Very Simple)

1. Convert **Hexadecimal** $\rightarrow$ **Binary** (4 bits per hex digit)
2. Convert **Binary** $\rightarrow$ **Octal** (group into 3 bits from right)

Hex $\rightarrow$ Binary (4-bit table reminder)

Hex.Decimal	Binary(8421)	Hex.Decimal	Binary(8421)
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Example 1

Convert $(2F)_{16}$ to Octal

Hex $\rightarrow$ Binary

2 $\rightarrow$ 0010

F $\rightarrow$ 1111

Binary = 00101111

Binary $\rightarrow$ Octal (group by 3 from right)

001 011 111

001 $\rightarrow$ 1, 011 $\rightarrow$ 3, 111 $\rightarrow$ 7

Answer:

$$(2F)_{16} = (137)_8$$

Example 2

Convert $(A3)_{16}$ to Octal

Hex $\rightarrow$ Binary

A $\rightarrow$ 1010

3 $\rightarrow$ 0011

Binary = 10100011

Binary $\rightarrow$ Octal

010 100 011

010 $\rightarrow$ 2, 100 $\rightarrow$ 4, 011 $\rightarrow$ 3

Answer:

$$(A3)_{16} = (243)_8$$

With Fraction

Convert $(A.5)_{16}$ to Octal

Hex $\rightarrow$ Binary

A $\rightarrow$ 1010

.5 $\rightarrow$.0101

Binary = 1010.0101

Binary $\rightarrow$ Octal

- Integer: 001 010 $\rightarrow$ 1, 2
- Fraction: 010 100 $\rightarrow$.2, .4

Answer:

$$(A.5)_{16} = (12.24)_8$$

Hexadecimal to octal:

Hexadecimal	Octal
$(5A8)_{16}$	$=0101\ 1010\ 1000(\text{Binary})$
	$=010\ 110101000(\text{Binary})$
Result	$=2\ 6\ 5\ 0(\text{Octal})$

Octal to Hexadecimal Conversion

For conversion of octal to Hexadecimal, first convert the given octal number to binary and then binary number to hexa decimal.

Steps

1. Convert **Octal** $\rightarrow$ **Binary** (3 bits per octal digit)
2. Convert **Binary** $\rightarrow$ **Hexadecimal** (group into 4 bits from right)

Octal $\rightarrow$ Binary (3-bit table reminder)

Octal	Binary(421)
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Example 3

Convert $(57)_8$ to Hexadecimal

Octal $\rightarrow$ Binary

$5 \rightarrow 101$

$7 \rightarrow 111$

Binary = **101111**

Binary $\rightarrow$ Hex (group by 4 from right)

0010 1111

0010 $\rightarrow$ 2, 1111 $\rightarrow$ F

Answer:

$$(57)_8 = (2F)_{16}$$

Example 4

Convert $(124)_8$ to Hexadecimal

Octal $\rightarrow$ Binary

1 $\rightarrow$ 001

2 $\rightarrow$ 010

4 $\rightarrow$ 100

Binary = **001010100**

Binary $\rightarrow$ Hex.Dec

0001 0101 00

$\rightarrow$ pad $\rightarrow$ 0001 0101 0000

0001 $\rightarrow$ 1, 0101 $\rightarrow$ 5, 0000 $\rightarrow$ 0

Answer:

$$(124)_8 = (150)_{16}$$

With Fraction

Convert $(36.5)_8$ to Hexadecimal

Octal $\rightarrow$ Binary

3 $\rightarrow$ 011

6 $\rightarrow$ 110

.5 $\rightarrow$.101

Binary = 011110.101

Binary $\rightarrow$ Hex. decimal

- Integer: 0011 1110 $\rightarrow$ 3E
- Fraction: .1010 $\rightarrow$. A

Answer:

$$(36.5)_8 = (3E.A)_{16}$$

Octal to Hexadecimal:

(May 2014)

Octal	Hexadecimal
=2 6 5 0	
= 010 110101000	= 0101 1010 1000 (Binary)
Result	=(5A8) ₁₆

DIGITAL CODES

We have BCD code, Excess-3 code, gray code

1. BINARY CODED DECIMAL (BCD) (or) (NATURAL BCD CODE) (or) 8421 BCD CODE BCD

BCD is a numeric code in which each digit of a decimal number is represented by a separate group of bits. The most common BCD code is 8-4-2-1 BCD, in which each decimal digit is represented by a 4-bit binary number. It is called 8-4-2-1 BCD because the weights associated with 4 bits are 8-4-2-1 from left to right. This means that, bit 3 has weight 8, bit 2 has weight 4, bit 1 has weight 2 and bit 0 has weight 1.

The following Table shows the 4-bit 8-4-2-1 BCD code used to represent a single decimal digit. The 8-4-2-1 BCD code is so widely used that it is common practice to refer to it simply as BCD.

Decimal	BCD Code			
Digit	8	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Problems on BCD Code

Problem 1

Convert the decimal number 29 into BCD code.

In BCD, each decimal digit is converted separately.

- $2 \rightarrow 0010$
- $9 \rightarrow 1001$

BCD of 29 = 0010 1001

Problem 2

Convert the decimal number 507 into BCD code.

Convert each digit:

- $5 \rightarrow 0101$

- $0 \rightarrow 0000$
- $7 \rightarrow 0111$

BCD of 507 = 0101 0000 0111

Problem 3

Convert (1001 0100)BCD into decimal number.

Split into 4-bit groups:

- $1001 \rightarrow 9$
- $0100 \rightarrow 4$

Decimal number = 94

Problem 4

Write the BCD representation of 86.

- $8 \rightarrow 1000$
- $6 \rightarrow 0110$

BCD of 86 = 1000 0110

Problem 5

Convert (0100 0011)BCD into decimal.

- $0100 \rightarrow 4$
- $0011 \rightarrow 3$

Decimal number = 43

Problem 6

Is (1010)BCD a valid BCD code? Explain.

Valid BCD codes are only from:

- 0000 to 1001 (0 to 9)

Since $1010 = 10$, it is not allowed in BCD.

(1010)BCD is INVALID

Problem 7

Write BCD code for decimal number 904.

- $9 \rightarrow 1001$
- $0 \rightarrow 0000$
- $4 \rightarrow 0100$

BCD of 904 = 1001 0000 0100

Problem 8

Convert decimal 123 into BCD.

- $1 \rightarrow 0001$
- $2 \rightarrow 0010$
- $3 \rightarrow 0011$

BCD of 123 = 0001 0010 0011

Problem 9

Convert (0110 1001)BCD into decimal.

- $0110 \rightarrow 6$
- $1001 \rightarrow 9$

Decimal number = 69

Excess-3 Code

Excess-3 code is a non-weighted binary code. It is obtained by adding 3 to each decimal digit, then converting the result to 4-bit binary.

Formula:

$$\text{Excess - 3 Code} = \text{Decimal Digit} + 3$$

Excess-3 Code Table

Decimal	Decimal + 3	Excess-3 Code(8421)
0	3	0011
1	4	0100
2	5	0101
3	6	0110
4	7	0111
5	8	1000
6	9	1001
7	10	1010
8	11	1011
9	12	1100

Example

Represent decimal 47 in Excess-3 code

- $4 + 3 = 7 \rightarrow 0111$
- $7 + 3 = 10 \rightarrow 1010$

Excess-3 of 47 = 0111 1010

Convert (367)₁₀ into its Excess-3 code.

Solution.	The decimal number is	3	6	7
	Add 3 to each bit	+3	+3	+3
	Sum	6	9	10

Converting the above sum into 4-bit binary equivalent, we have a 4-bit binary equivalent of 0110 1001 1010

Hence, the Excess-3 code for $(367)_{10} = 0110 \ 1001 \ 1010$

Gray code

Gray code is a binary code in which only one bit changes at a time when moving from one number to the next.

Because only one bit changes, Gray code **reduces errors** in digital systems such as **encoders and ADCs**.

Conversion of binary code to gray code:

1. The Most Significant Bit (MSB) of Gray code is same as the MSB of Binary code.
2. Each remaining Gray bit is obtained by taking the **XOR** of two adjacent binary bits.

(i) Binary to Gray code conversion:

If an n-bit binary number is represented by $B_n B_{n-1} \dots B_1$ and its Gray code equivalent by $G_n G_{n-1} \dots G_1$, where B_n and G_n are the MSBs, then the Gray code bits are obtained from the binary code as follows.

$G_n = B_n$	$G_{n-1} = B_n \oplus B_{n-1}$	$G_{n-2} = B_{n-1} \oplus B_{n-2} \dots$	$G_1 = B_2 \oplus B_1$
-------------	--------------------------------	--	------------------------

Where the symbol $\oplus$ stands for the Exclusive OR (X-OR) operation explained below.
The conversion procedure is as follows:

If the binary number is:

$$B_n \ B_{n-1} \ \dots \ B_2 \ B_1$$

Then the Gray code is:

- $G_n = B_n$
- $G_{n-1} = B_n \oplus B_{n-1}$
- $G_{n-2} = B_{n-1} \oplus B_{n-2}$
- ...
- $G_1 = B_2 \oplus B_1$

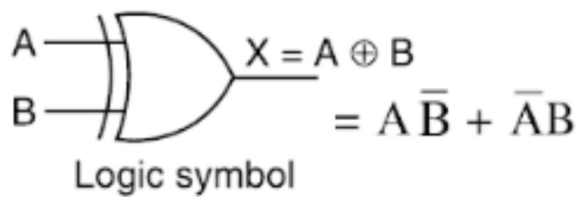
Where $\oplus$ means XOR (Exclusive-OR).

Gate Operation Required

XOR (Exclusive-OR) gate is used to convert Binary $\rightarrow$ Gray code.

The exclusive OR gate is sometimes referred to as the "Odd but not the even gate". It is often shortened as "XOR gate". The logic diagram is shown in Fig. with its Boolean expression. The symbol $\oplus$ means the terms are XORed together.

It has 2 inputs & only 1 output. It assumes output as 1 when inputs are not equal and it is called anti-coincidence gate or inequality detector. The logic symbol and truth tables are shown in the fig.



(a)

Inputs		Output
A	B	$X = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

(b) Truth table

Ex:-

① Convert Binary code $(1001)_2$ to The Gray code

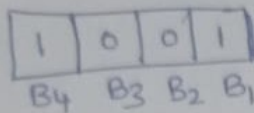


fig: Binary code

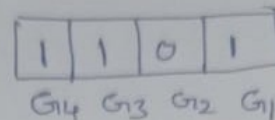


fig: Gray code

$$G_n = B_n$$

$$G_{n-1} = B_n \oplus B_{n-1}$$

$$G_{n-2} = B_{n-1} \oplus B_{n-2}$$

$$G_4 = B_4$$

$$G_{4-1} = G_3 = B_4 \oplus B_3$$

$$= 1 \oplus 0$$

$$= (1)(0) + (0)(1)$$

$$= (0 \times 0) + (1 \times 1)$$

$$= 1$$

$$\begin{aligned} \text{XOR} &= A \oplus B \\ &= \bar{A}B + A\bar{B} \end{aligned}$$

$$\begin{cases} 1 \times 0 \\ 0 \times 1 \end{cases}$$

$$G_{3-1} = G_2 = B_3 \oplus B_2$$

$$0 \oplus 0 =$$

$$\overline{A}B + A\overline{B} = (\overline{0}) \times 0 + (0) \times \overline{0}$$

$$= 1 \times 0 + 0 \times 1$$

$$= 0 + 0$$

$$= 0$$

$$G_{2-1} = G_1 = B_2 \oplus B_1$$

$$= 0 \oplus 1$$

$$\overline{A}B + A\overline{B} = (\overline{0}) \times 1 + (0) \times \overline{1}$$

$$= (1 \times 1) + (0 \times 0) = 1 + 0 = 1$$

Ans: Gray code = 1101

For example :-

Convert the binary 1001 to the Gray code.

Solution :-

$$\begin{array}{ccccccc} \text{Binary} & \rightarrow & 1 & \xrightarrow{\oplus} & 0 & \xrightarrow{\oplus} & 0 & \xrightarrow{\oplus} & 1 \\ & & \downarrow & & \downarrow & & \downarrow & & \downarrow \\ \text{Gray} & \rightarrow & 1 & & 1 & & 0 & & 1 \end{array}$$

The gray code is 1101

Decimal Number	Binary Code	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111

11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Gray Code to Binary Conversion

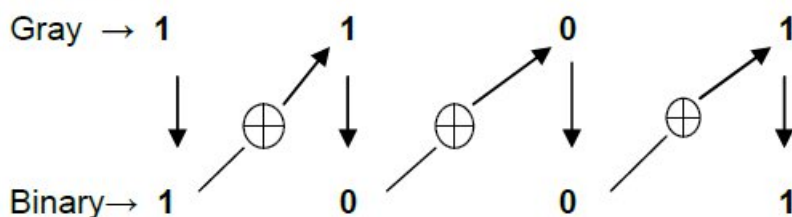
What is Gray $\rightarrow$ Binary Conversion?

It is the process of converting a Gray code number back into its equivalent binary number.

1. The MSB (first bit) of Binary = MSB of Gray
2. Each next Binary bit = XOR of previous Binary bit and current Gray bit
3. Performs XOR Gate operation

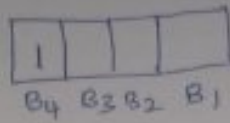
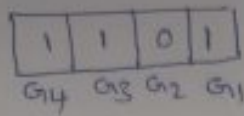
Convert the Gray code 1101 to the binary.

Solution :-



The binary code is **1001**

Convert The Gray code 1101 to The Binary



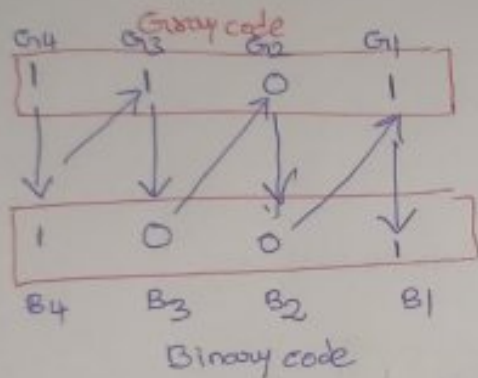
$$B_4 = G_4$$

$$B_3 = B_4 \oplus G_3$$

$$= 1 \oplus 1$$

$$= 0$$

XOR Gate
 $A \oplus B = \bar{A}B + A\bar{B}$



$$B_2 = B_3 \oplus G_2$$

$$= 0 \oplus 0$$

$$= 0$$

$$B_1 = B_2 \oplus G_1$$

$$= 0 \oplus 1$$

$$= 1$$

$\bar{A}B + A\bar{B}$

LOGIC GATES (V.IMP Question)

Logic gates are the fundamental building blocks of digital systems. A gate is defined as a logic circuit with one or more inputs and only one output which implement a desired logic function.

The interconnection of gates to perform a variety of logical operations is called logic design. The logic gates are usually embedded in LSI and VLSI circuits.

The Inputs and outputs of logic gates can occur only in two levels. These two levels are termed HIGH and LOW or TRUE and FALSE or ON and OFF or simply 1 and 0.

The terminology used for logic levels is shown in the fig.

<i>Logic 0</i>	<i>Logic 1</i>
False	True
Off	On
Low	High
No	Yes
Open switch	Close switch

TYPES OF LOGIC GATES

The logic gates are of three types.

- 1) **Basic gates:** AND, OR, NOT
- 2) **Universal gates:** NAND, NOR
- 3) **Desired gates:** EX-OR, EX-NOR

AND Gate:

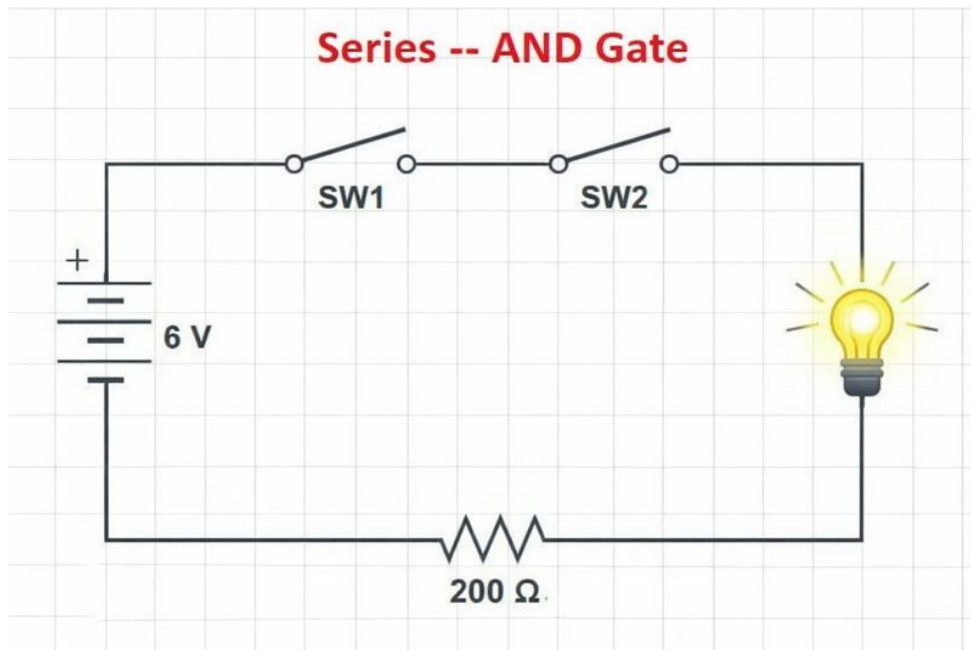
Definition

The AND gate gives output 1 only when ALL inputs are 1.

Boolean Expression

$$Y = A \cdot B$$

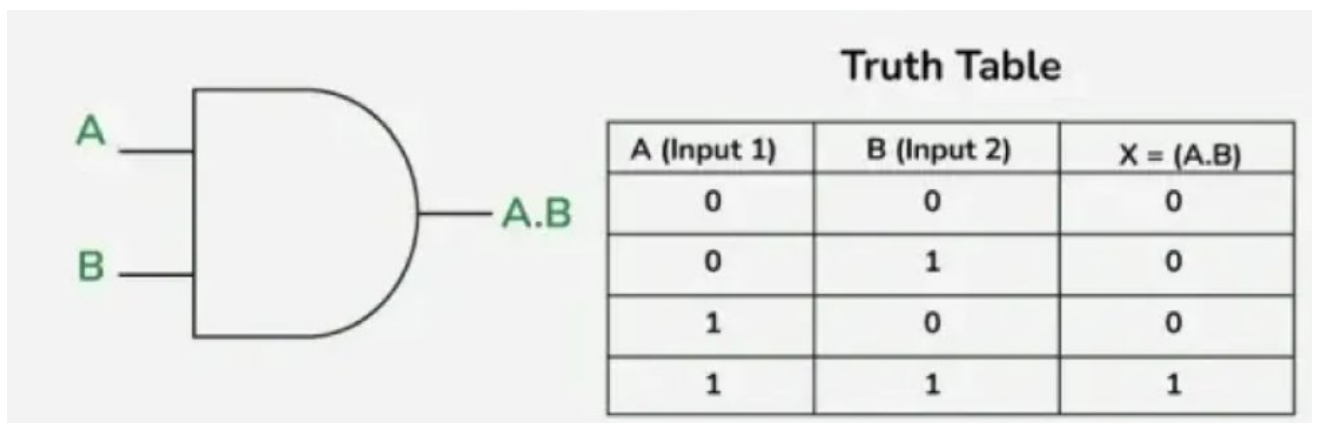
An AND gate is used to perform logical Multiplication of binary input. The Output state of the AND gate will be high (1) if both the input is high (1), else the output state will be low (0) if any of the input is low (0).



Switches in SERIES connection

- Two switches A and B are connected in series with a bulb.
- Bulb glows only when both switches are ON.

Switch A	Switch B	Bulb(output)
OFF	OFF	OFF
OFF	ON	OFF
ON	OFF	OFF
ON	ON	ON



OR Gate:

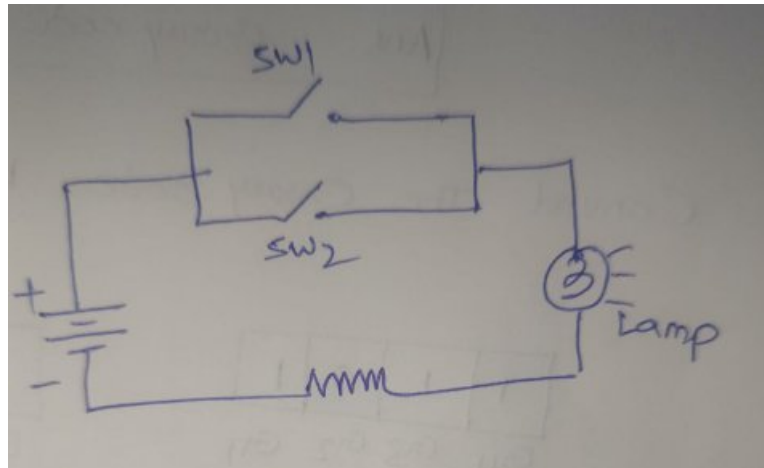
Definition

The OR gate gives output 1 if ANY one input is 1.

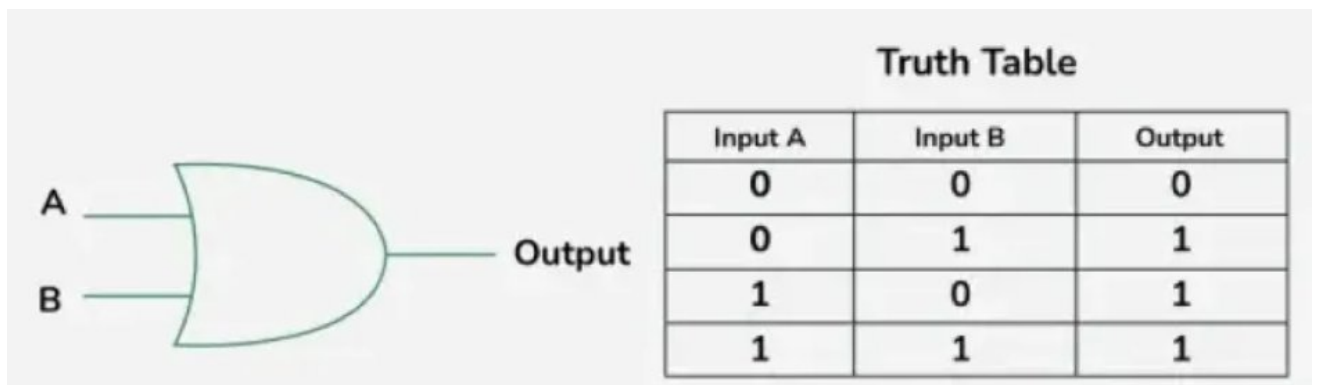
Boolean Expression

$$Y = A + B$$

When all of the inputs to the OR gate are low or logic 0, the output of it is low or logic 0.



Switch A	Switch B	Bulb(output)
OFF	OFF	OFF
OFF	ON	ON
ON	OFF	ON
ON	ON	ON

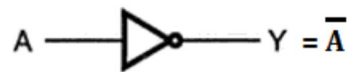


NOT GATE:

It is represented by '—'(bar). It is also called an Inverter or Buffer. It has only one input & one output in which the output is always the compliment of its input.

The output assumes logic 1 when input is logic 0 & output assume logic 0 when input is logic 1.

The logic symbol and truth table of NOT gate are shown in the fig.



(a) Logic symbol

A	Y
0	1
1	0

(b) Truth table

UNIVERSAL GATES (NAND & NOR Gates) (V.IMP Question)

The NAND and NOR gates are widely used and are readily available in most integrated circuits. A major reason for the widespread use of these gates is that they are both UNIVERSAL gates, universal in the sense that both can be used for AND operators, OR operators, as well as Inverter. Thus, we see that a complex digital system can be completely synthesized using only NAND gates or NOR gates.

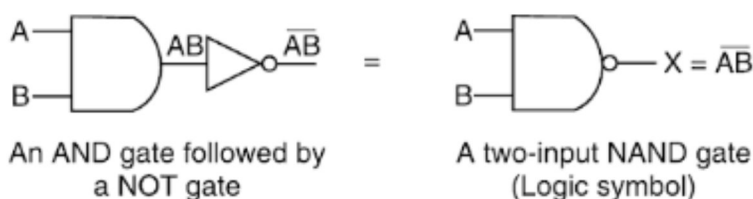
NAND GATE:(AND + NOT)

The NAND Gate is another type of Universal logic gate. The NAND gate or "Not AND" is the combination of two basic logic gates AND gate and the NOT gate connected in series. It takes two or more inputs and gives only one output.

The output of the NAND gate will give result high (1) when either of its input is high (1) or both of its input are low (0). In simple, it performs the inverted operation of AND gate.

NAND gate is a combination of AND & NOT gates. The little invert bubble (small circle) on the right end of the symbol means to invert the output of AND.

The Boolean Expression of NAND Gate is as follows



(a)

Truth table		
Inputs		Output
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

(b)

NAND gate produces a low or logic 0 output only when its all inputs are high or logic 1.

NOR Gate:(OR+ NOT)

Definition

A NOR gate is a digital logic gate that produces an output LOW (0) if any one or more inputs are HIGH (1). The output is HIGH (1) only when all inputs are LOW (0).

In simple words:

NOR = NOT of OR

Boolean Expression

For a 2-input NOR gate with inputs **A** and **B**: $Y = \overline{A + B}$

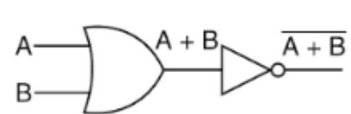
Where:

- $+$ represents **OR**
- $\overline{}$ (bar) represents **NOT**



Truth Table

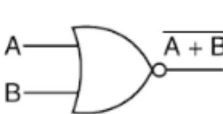
Input A	Input B	$0 = (A + B)'$
0	0	1
0	1	0
1	0	0
1	1	0



An OR gate followed by a NOT gate

(a)

$=$



A two-input NOR gate (logic symbol)

(b) Truth table

Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

The unique property of NOR gate is that it produces an output of logic '1' if and only if all its inputs are logic '0'.

XOR (Exclusive-OR) Gate

Definition

An XOR (Exclusive-OR) gate is a logic gate whose output is HIGH (1) only when the inputs are different.

If both inputs are the same (both 0 or both 1), the output is LOW (0).

In simple words:

XOR = Different inputs → Output 1

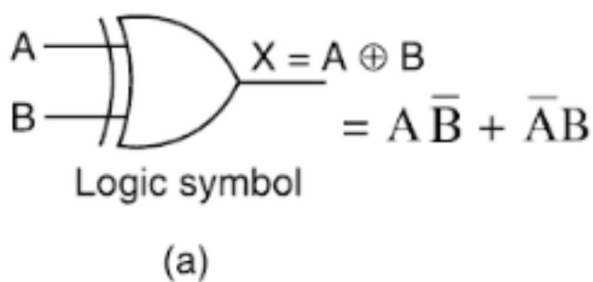
Boolean Expression

For a 2-input XOR gate with inputs **A** and **B**:

$$Y = A \oplus B = AB + \bar{A}\bar{B}$$

The exclusive OR gate is sometimes referred to as the “Odd but not the even gate”. It is often shortened as “XOR gate”. The logic diagram is shown in Fig. with its Boolean expression. The symbol $\oplus$ means the terms are XORED together.

It has 2 inputs & only 1 output. It assumes output as 1 when inputs are not equal and it is called anti-coincidence gate or inequality detector. The logic symbol and truth tables are shown in the fig.



Inputs		Output
A	B	$X = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

(b) Truth table

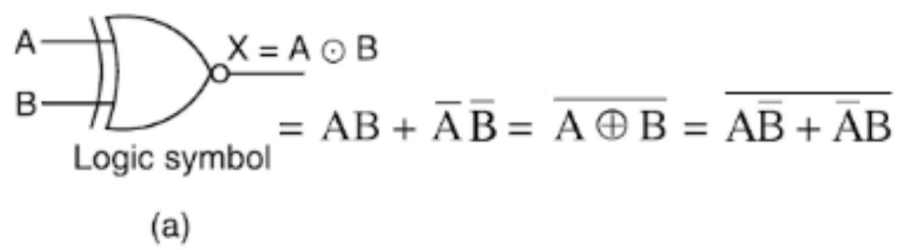
XNOR Gate:(XOR + NOT)

An X-NOR gate is the combination of an XNOR & NOT gate

An X-NOR gate is a two input, one output logic circuit.

The output is logic 1 only when both the inputs

The output is logic 0 when one of the inputs is logic 0 and other is 1.



$$= AB + \bar{A}\bar{B} = \overline{A \oplus B} = \overline{A\bar{B} + \bar{A}B}$$

Inputs		Output
A	B	$X = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

(b) Truth table

Name	Graphic symbol	Algebraic function	Truth table															
AND		$F = x \cdot y$	<table> <tr><th>x</th><th>y</th><th>F</th></tr> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = x + y$	<table> <tr><th>x</th><th>y</th><th>F</th></tr> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$F = x'$	<table> <tr><th>x</th><th>F</th></tr> <tr><td>0</td><td>1</td></tr> <tr><td>1</td><td>0</td></tr> </table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
Buffer		$F = x$	<table> <tr><th>x</th><th>F</th></tr> <tr><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td></tr> </table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	
NAND		$F = (xy)'$	<table> <tr><th>x</th><th>y</th><th>F</th></tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = (x + y)'$	<table> <tr><th>x</th><th>y</th><th>F</th></tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	<table> <tr><th>x</th><th>y</th><th>F</th></tr> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	<table> <tr><th>x</th><th>y</th><th>F</th></tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Boolean Algebra

Boolean Algebra is a type of mathematics used in digital electronics to work with logic gates and binary values.

It deals with only two values:

- $0 \rightarrow \text{FALSE} / \text{LOW} / \text{OFF}$
- $1 \rightarrow \text{TRUE} / \text{HIGH} / \text{ON}$

Basic Laws (Theorems) of Boolean Algebra

Law 1: Identity Law

Expression	Result
$A + 0$	A
$A \cdot 1$	A

:

- OR with 0 $\rightarrow$ no change
- AND with 1 $\rightarrow$ no change

Law 2: Null Law

Expression	Result
$A + 1$	1
$A \cdot 0$	0

Meaning: , - - - -

Proof: a)

$$\begin{aligned}A + 1 &= 1 \\A + 1 &= 1 \cdot (A + 1) \\&= (A + A') (A + 1) \quad (\text{V postulate}) \\&= A + A' \cdot 1 \quad (\text{IV Postulate}) \\&= A + A' \\&= 1\end{aligned}$$

b) $A \cdot 0 = 0$

$$\begin{aligned}A \cdot 0 &= A \cdot 0 + 0 \\&= A \cdot 0 + A \cdot A'\end{aligned}$$

$$\begin{aligned}&= A \cdot (0 + A') \\&= A \cdot A' \\&= 0 \quad (\because \text{Duality})\end{aligned}$$

- AND with 0 $\rightarrow$ always 0

Law 3: Idempotent Law

Expression	Result
$A + A$	A
$A \cdot A$	A

$$\begin{aligned}
 A \cdot A &= A \cdot A + 0 \\
 &= A \cdot A + A \cdot A' \\
 &= A(A + A') \\
 &= A \cdot 1 \\
 &= A
 \end{aligned}$$

Law 4: Complement Law

Expression	Result
$\overline{A + A}$	1
$\overline{A \cdot A}$	0

Law 5: Commutative Law

Law 1: $A + B = B + A$

Expression	Same as
$A + B$	$B + A$
$A \cdot B$	$B \cdot A$

Proof

A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	1

=

B	A	B + A
0	0	0
0	1	1
1	0	1
1	1	1

Law 2: $A \cdot B = B \cdot A$

Proof

A	B	A . B
0	0	0
0	1	0
1	0	0
1	1	1

=

B	A	B . A
0	0	0
0	1	0
1	0	0
1	1	1

Law 6: Associative Law

Expression	Same as
$(A + B) + C$	$A + (B + C)$
$(A \cdot B) \cdot C$	$A \cdot (B \cdot C)$

There are 2 associative laws.

Law 1: $(A + B) + C = A + (B + C)$

Proof

A	B	C	A+B	(A+B)+C
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

=

A	B	C	B+C	A+(B+C)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	0	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Law 2: $(A \cdot B) \cdot C = A \cdot (B \cdot C)$

Proof

A	B	C	AB	(AB)C
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	1	0
1	1	1	1	1

=

A	B	C	B.C	A(B.C)
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Law 7: Distributive Law

Expression	Result
$A(B + C)$	$AB + AC$
$A + BC$	$(A + B)(A + C)$

Law 1: $A(B + C) = AB + AC$ **Proof**

A	B	C	B+C	A(B+C)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

=

A	B	C	AB	AC	A+(B+C)
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	0	0	0
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	1	1

Law 2: $A + BC = (A+B)(A+C)$ **Proof** RHS = $(A+B)(A+C)$

$$= AA + AC + BA + BC$$

$$= A + AC + AB + BC$$

$$= A(1 + C + B) + BC$$

$$= A \cdot 1 + BC \quad (1 + C + B = 1 + B = 1)$$

$$= A + BC$$

$$= \text{LHS}$$

Law 8: Absorption Law

Expression	Result
$A + AB$	A
$A(A + B)$	A

$$A + AB = A \cdot 1 + AB$$

$$= A(1 + B) \quad A(A + B) = A \cdot A + AB$$

$$= A \cdot 1 \quad = A + AB$$

$$= A \quad = A$$

+

9. Absorption Laws:-

There are two laws:

Law 1: $A + A \cdot B = A$

Proof

$$A + A \cdot B = A(1 + B) = A \cdot 1 = A$$

A	B	AB	A+AB
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

Law 2: $A(A + B) = A$

Proof

$$A(A + B) = A \cdot A + A \cdot B = A + AB = A(1 + B) = A \cdot 1 = A$$

A	B	A+B	A(A+B)
0	0	0	0
0	1	1	0
1	0	1	1
1	1	1	1

Law 9: Double Negation Law

Expression	Result
$\overline{\overline{A}}$	A

De Morgan's Theorems (VERY IMPORTANT)

Theorem 1

12. De Morgan's Theorem:-

De Morgan's theorem represents two laws in Boolean algebra.

Law 1: $\overline{A + B} = \overline{A} \cdot \overline{B}$

Proof

A	B	A + B	$\overline{A + B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

=

A	B	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	0	0	0

This law states that the complement of a sum of variables is equal to the product of their individual complements.

Theorem 2

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

Law 2: $\overline{A \cdot B} = \overline{A} + \overline{B}$

Proof

A	B	$A \cdot B$	$\overline{A \cdot B}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

=

A	B	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$
0	0	1	1	1
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0

This law states that the complement of a product of variables is equal to the sum of their individual complements.

3) Reduce the expression

$$f = A[B + \overline{C}(\overline{AB} + \overline{AC})].$$

SOL:

The given expression is

Demorganize $\overline{AB} + \overline{AC}$

Demorganize $\overline{AB}$ and $\overline{AC}$

Multiply $(\overline{A} + \overline{B})(\overline{A} + C)$

Simplify

Simplify

Simplify

Simplify

Simplify

$$f = A[B + \overline{C}(\overline{AB} + \overline{AC})]$$

$$= A[B + \overline{C}(\overline{A} \overline{B} + \overline{A} \overline{C})]$$

$$= A[B + \overline{C}(\overline{A} + \overline{B})(\overline{A} + C)]$$

$$= A[B + \overline{C}(\overline{A} \overline{A} + \overline{A} \overline{C} + \overline{B} \overline{A} + \overline{B} \overline{C})]$$

$$= A(B + \overline{C} \overline{A} + \overline{C} \overline{A} \overline{C} + \overline{C} \overline{B} \overline{A} + \overline{C} \overline{B} \overline{C})$$

$$= A(B + \overline{C} \overline{A} + 0 + \overline{C} \overline{B} \overline{A} + 0)$$

$$= AB + A \overline{C} \overline{A} + A \overline{C} \overline{B} \overline{A}$$

$$= AB + 0 + 0$$

$$= AB$$

4) Reduce the expression

$$f = (B + BC)(B + \overline{B}C)(B + D).$$

The given expression is

Multiply the first two terms

Reduce

Factor

Reduce

Expand

Simplify

$$f = (B + BC)(B + \overline{B}C)(B + D)$$

$$= (BB + BCB + B \overline{B}C + BC \overline{B}C)(B + D)$$

$$= (B + BC + 0 + 0)(B + D)$$

$$= B(1 + C)(B + D)$$

$$= B(B + D)$$

$$= BB + BD$$

$$= B(1 + D) = B$$

5) Simplify the expression

$$AB + \overline{AC} + A\overline{B}C (AB + C)$$

SOL:

$$= AB + \overline{AC} + AA\overline{B}BC + A\overline{B}CC$$

$$= AB + \overline{AC} + A\overline{B}CC$$

$$= AB + \overline{AC} + A\overline{B}C$$

$$= AB + \overline{A} + \overline{C} + A\overline{B}C$$

$$= \overline{A} + B + \overline{C} + A\overline{B}C$$

$$= \overline{A} + A\overline{B}C + B + \overline{C}$$

$$= \overline{A} + \overline{B}C + B + \overline{C}$$

$$= \overline{A} + B + \overline{C} + \overline{B}C$$

$$= \overline{A} + B + \overline{C} + \overline{B}$$

$$= \overline{A} + \overline{C} + 1$$

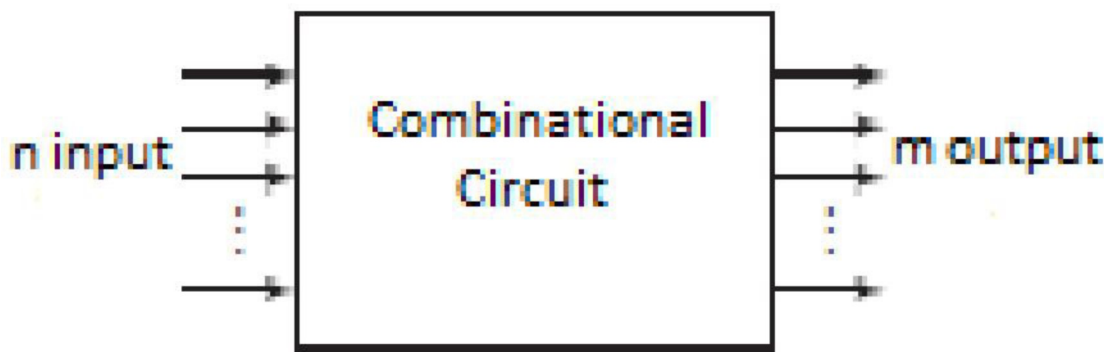
$$= 1$$

COMBINATIONAL LOGIC CIRCUITS

A combinational logic circuit is a digital circuit in which the output depends only on the present inputs.

- ✓ There is NO memory in combinational circuits.
- ✓ Output changes immediately when input changes.
- ✓ A combinational circuit performs an operation that can be specified logically by a set of Boolean functions.
- ✓ It consists of an interconnection of logic gates. Combinational logic gates react to the values of the signals at their inputs and produce the value of the output signal, transforming binary information from the given input data to a required output data.
- ✓ A block diagram of a combinational circuit is shown in the below figure.

A combinational logic circuit is a digital circuit whose output is a direct function of the present input values only.



types of Combinational Logic Circuits

The most important types are:

1. Adder
2. Subtractor
3. Multiplexer (MUX)
4. Demultiplexer (DEMUX)
5. Encoder
6. Decoder
7. Comparator

ADDERS

Digital computers perform various arithmetic operations and the most basic arithmetic operation is the addition of two binary digits .i.e, 4 basic possible operations are:

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10_2$$

The first three operations produce a sum whose length is one digit, but when augends and addend bits are equal to 1, the binary sum consists of two digits. The higher significant bit of this result is called a **carry**.

A combinational circuit that performs the addition of two bits is called a half-adder.

One that performs the addition of 3 bits (two significant bits & previous carry) is called a full adder & two half adders can be employed to implement a full-adder.

HALF ADDER

Function

Adds two binary bits. And produces two outputs **sum and carry**

Inputs

- A, B

Outputs

- Sum (S)
- Carry (C)

Formulas

$$S = A \oplus B \quad S = A\bar{B} + \bar{A}B = A \oplus B$$

Where S is the **SUM**. sum is obtained from **XOR** gate operation

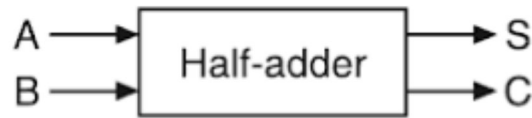
$$C = A \cdot B$$

Where C is **CARRY** which is obtained from **AND** gate operation

A Half Adder is a combinational circuit with two binary inputs (augends and addend bits) and two binary outputs (sum and carry bits.) It adds the two inputs (A and B) and produces the sum (S) and the carry (C) bits. It is an arithmetic circuit used to perform the arithmetic operation of addition of two single bit words. The truth table and block diagram of half adder are shown in the following figs.

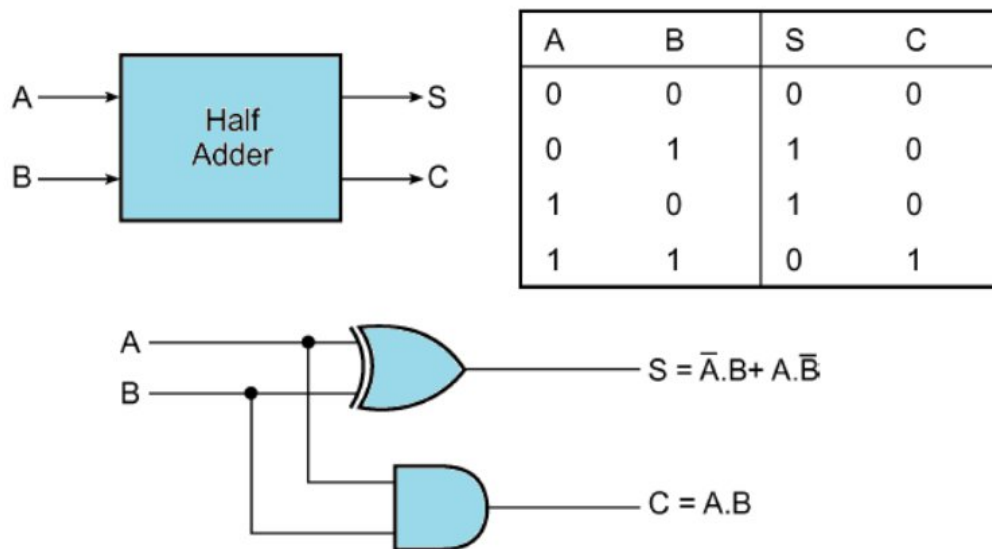
Inputs		Outputs	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

(a) Truth table



(b) Block diagram

A half-adder can be realized by using one X-OR gate and one AND gate as shown in fig.



FULL ADDER

A Full-adder is an arithmetic circuit block that can be used to add three bits and produce two outputs SUM and CARRY.

Function

Adds three bits (A, B, Carry-in).

Formulas

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + BC_{in} + AC_{in}$$

A Full-adder is a combinational circuit that adds two bits and a carry and outputs a sum bit and a carry bit. The full-adder adds the bits A and B and the carry from the previous column called the carry-in C_{in} and outputs the sum bit S and the carry bit called the carry-out C_{out} . The variable S gives the value of the least significant bit of the sum. The variable C_{out} gives the output carry.

The block diagram and truth table of full adder are shown in the following fig. The eight rows under the input variables designate all possible combinations of 1s and 0s that these variables may have. The 1s and 0s for the output variables are determined from the arithmetic sum of the

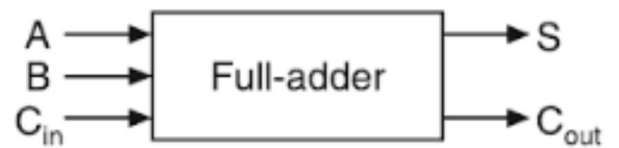
input bits. When all the bits are 0s, the output is 0. The S output is equal to 1 when only input is equal to 1 or when all the inputs are equal to 1. The Cout has a carry of 1 if two or three inputs are equal to 1.

$0+0+0=0\ 0$
 $0+0+1=0\ 1$
 $0+1+0=01$
 $0+1+1=10$
 $1+0+0=01$
 $1+0+1=10$

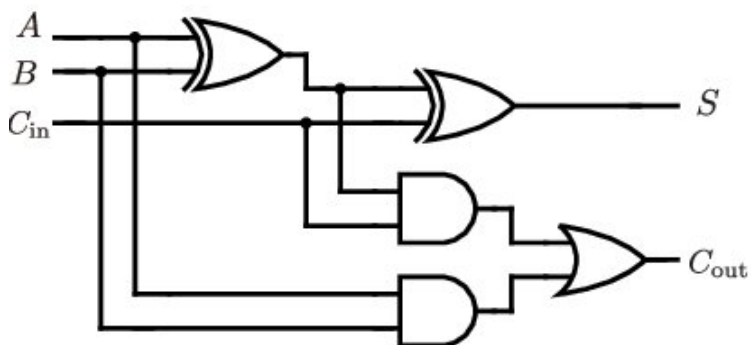
$1+1=10$ IN which right 0 is sum and left 1 is carry

Inputs			Sum	Carry
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(a) Truth table



(b) Block diagram

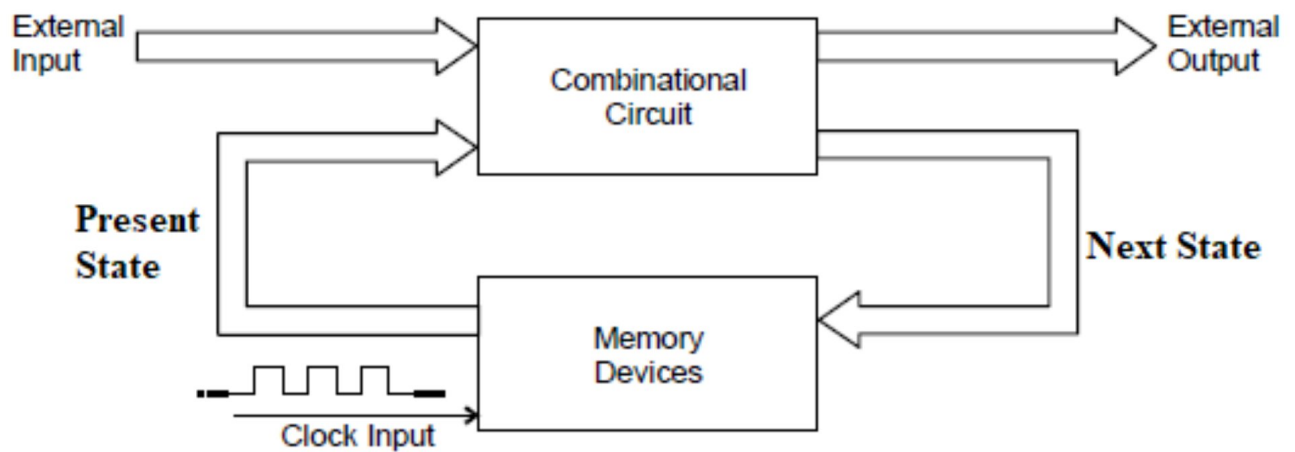


Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$1+1 = 10$
 Sum=0
 Carry=1
 $1+1+1=10+1=11$
 Sum=1
 Carry=1

SEQUENTIAL LOGIC CIRCUITS

A sequential logic circuit is one whose outputs depend not only on its current inputs, but also on the past sequence of inputs.



Comparison Between Combinational and Sequential Circuits

Feature	Combinational Circuits	Sequential Circuits
Definition	Output depends only on present inputs	Output depends on present inputs and past outputs
Memory	No memory element	Has memory
Storage device	Not required	Uses flip-flops / latches
Feedback path	No feedback	Feedback present
Clock signal	Not required	Required (synchronous type)
Time dependency	Output changes immediately with input	Output changes according to clock or state
Complexity	Simple design	More complex design
Speed	Faster	Comparatively slower
Reliability	High	High (synchronous), low

Feature	Combinational Circuits	Sequential Circuits
		(asynchronous)
Examples	Adders, Subtractors, MUX, Decoder, Encoder	Flip-flops, Counters, Registers
Applications	Arithmetic & logic operations	Memory, control and timing circuits

Combinational circuits do not have memory and their output depends only on present inputs. Sequential circuits contain memory elements and their output depends on both present input and previous state. Combinational circuits do not require a clock signal, whereas sequential circuits usually operate with a clock. Examples of combinational circuits include adders and multiplexers, while sequential circuits include flip-flops and counters.

Applications of Sequential Circuits

- Counters
- Registers
- Digital clocks
- Memory units
- Control systems

FLIP – FLOPS

A flip-flop is a basic memory element in digital electronics that can store one bit of information (either 0 or 1).

It can store 1 bit of data

That bit can be 0 or 1

It remembers its value until we change it

It is a sequential logic circuit whose output depends on:

- Present input, and
- Previous output (stored value)

Inputs(two)

Set

Reset

0 0

0 1

1 0

1 1

Outputs(two)

Q and Q'

CLOCK SIGNAL=1

Why Do We Need Flip-Flops?

Flip-flops are used to:

- Store data (memory)
- Build counters
- Build registers
- Design CPUs, RAM, timers, digital clocks

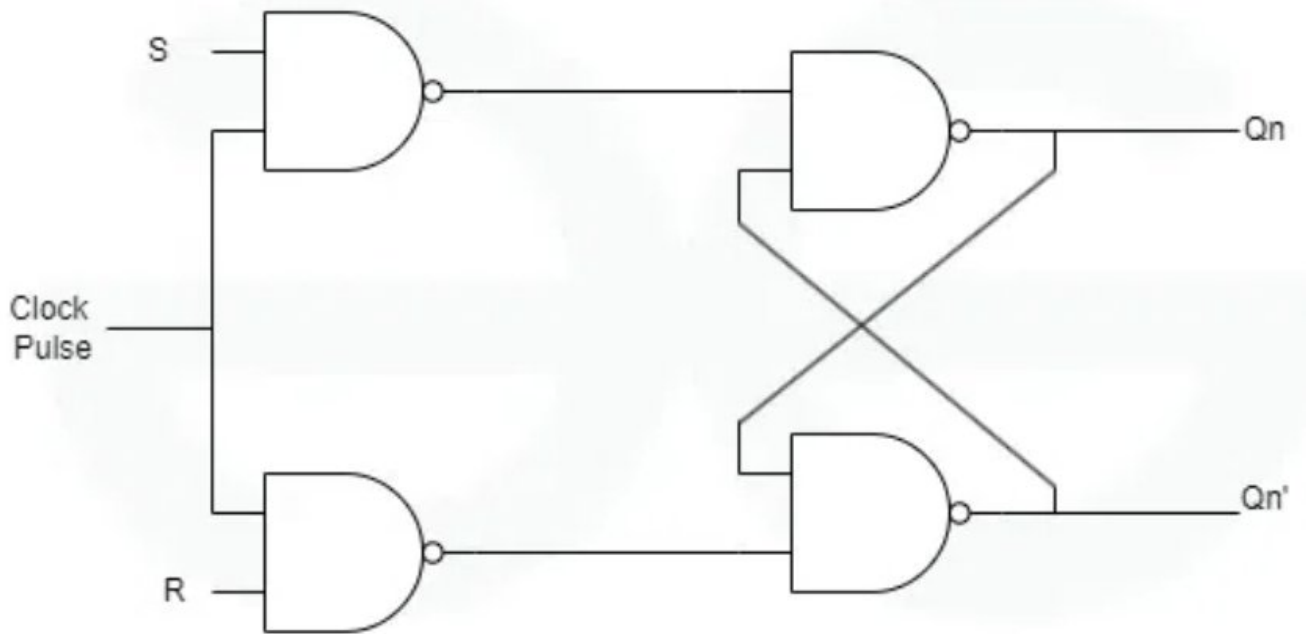
Without flip-flops, computers cannot remember anything.

Applications of Flip- flops

Types of Flip-Flops

1. SR Flip-Flop
2. JK Flip-Flop
3. D Flip-Flop
4. T Flip-Flop

SR Flip-Flop



Inputs:

- **S** → Set
- **R** → Reset
- **Clock**
- Outputs: Q and $\bar{Q}$

Truth Table:

S	R	Q (Next State)	Meaning
0	0	No change	Memory
0	1	0	Reset
1	0	1	Set
1	1	Invalid	Not allowed

S	R	Q	Q'
0	0	MEMORY	MEMORY
0	1		
1	0		
1	1	INVALID	INVALID

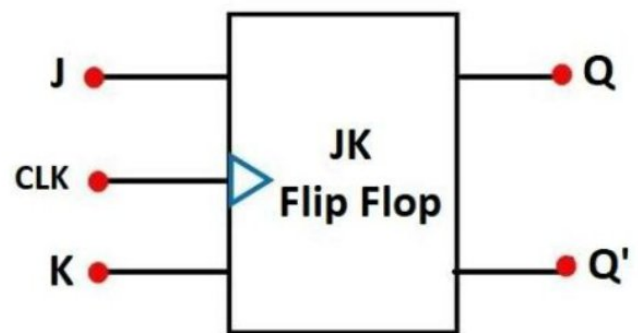
JK Flip-Flop

To remove the invalid state, JK flip-flop was invented.

◆ Inputs:

- J (like Set)
- K (like Reset)
- Clock

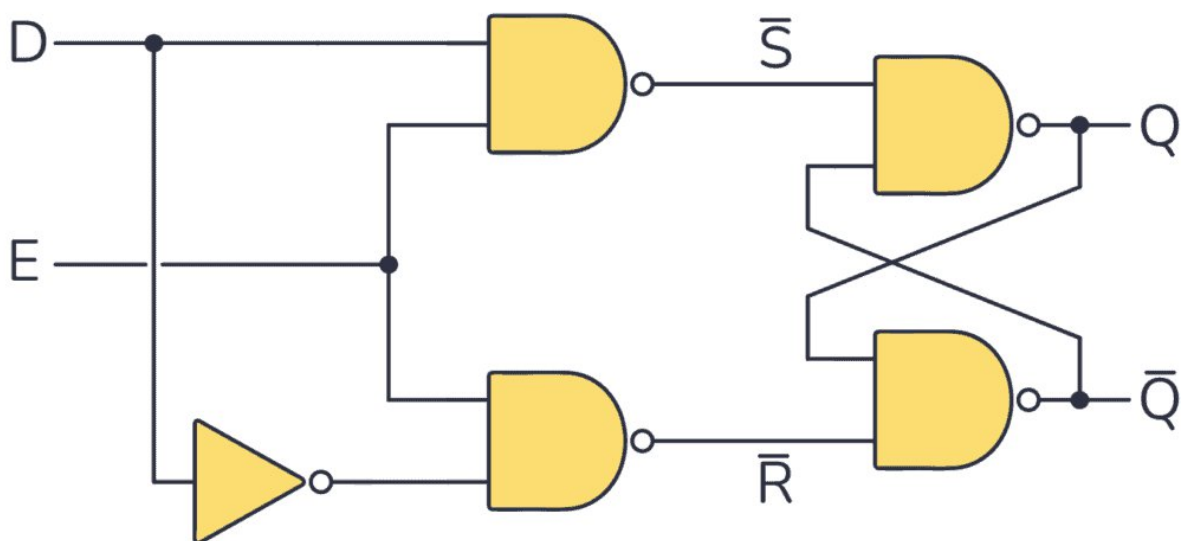
C	J	K	Q	Q'
HIGH	0	0	Latch	Latch
HIGH	0	1	0	1
HIGH	1	0	1	0
HIGH	1	1	Toggle	Toggle
LOW	0	0	Latch	Latch
LOW	0	1	Latch	Latch
LOW	1	0	Latch	Latch
LOW	1	1	Latch	Latch



hackatronic.com

D Flipflop

A D (Data or Delay) Flip-Flop is a clocked sequential circuit that stores one bit of data. It transfers the value present at the D input to the output Q only at the active edge of the clock and holds that value until the next clock edge.



inputs:

- D → Data input
- CLK → Clock input

Outputs:

- Q → Stored output
- $\bar{Q}$ → Complement of Q

Working Principle

The operation of a D flip-flop is very simple:

At the moment of the active clock edge, the output Q becomes equal to the input D.

Step-by-step explanation:

1. When clock is inactive, the flip-flop ignores input D.
2. When the active clock edge occurs (↑ rising edge or ↓ falling edge):
 - The value present at D is sampled
 - That value is transferred to Q
3. After the clock edge:
 - Output Q remains constant
 - Changes in D do not affect Q until the next clock edge

Hence, it is called a delay flip-flop because the input appears at the output one clock pulse later.

Truth Table

Clock Edge D Q (Next State)

↑ (Active) 0 0

↑ (Active) 1 1

No edge X Q (No change)

Q follows D only at the clock edge

Characteristic Equation

$$Q_{n+1} = D$$